Python for Data Analysis & Automation

Exercise Overview (60 min)

Abdullah Al Bashit, PhD | Dept. of Bioengineering, Northeastern University | March 27, 2026

Files Provided

- **`qPCR_pipeline.ipynb`** — Jupyter notebook with pre-written + blank cells
- **`Sample_Data.ods`** — qPCR data with **hidden issues for you to find!**

Legend: ✍ = Write your answer `# YOUR CODE HERE` = Fill in the code

Exercise Steps

Min	Task	Code (in notebook)	What to Look For
Part A: Python Basics — Execute & Learn (8 min)			
0:00	Open the notebook	`# Open qPCR_pipeline.ipynb`	Pre-written + blank cells
0:02	Run: imports	`import pandas as pd import numpy as np import matplotlib.pyplot as plt`	No errors = success
0:04	Run: variables, lists	`sample = "Cell_A" np.mean([22.0, 22.1, 7.8])`	str, float, list
0:05	Run: DataFrames	`df = pd.DataFrame({...}) df.head()`	See table output
0:06	Run: filtering rows	`example[example["Gene"] =="Target"]`	Filter keeps matching rows
0:07	Run: filter → column → mean	`example[...]["CT"].mean()`	Remember this pattern!
Part B: Read & Inspect qPCR Data (10 min)			
0:08	Load data file	`df = pd.read_excel( "Sample_Data.ods", engine="odf")`	48 rows × 13 cols
0:10	Inspect	`print(df.shape) df.head(12)`	Sample, Gene, CT
0:12	Check Gene column	`print(df["Gene"].unique())`	Should be 2...
✍	***CHALLENGE #1: df["Gene"].unique() returns 3 values. What is the extra one? Why?***		
✍	***Your answer:*** _____		
0:14	Investigate typo	`print(df["Gene"] .value_counts())`	Spacing issue!
0:15	Fix typo	`df["Gene"] = df["Gene"] .str.strip() .str.replace(...)`	Now 2 values ✓
0:16	Average tech reps	`avg = df.groupby(...) .mean().reset_index()`	48 → 24 rows
0:17	Split + merge	`target = avg[..."Target"] hk = avg[..."House..."] piv = target.merge( hk, on="Sample")`	Filter, filter, merge → 12 rows
Part C: Visualize — Spot the Errors! (10 min)			
0:18	Add group column	`piv["group"] = ...`	Control, Cond 1-3
0:19	Scatter: CT by condition	`ax.scatter(...) plt.show()`	One Target CT off in Cond 2

✍	***CHALLENGE #2: One Cond 2 point is far from the rest. Which replicate? ct_target = ?***		
✍	***Your answer: Cond 2 Replicate ____ , ct_target = _____***		
0:22	Plot HK stability	`# Plot ct_hk by condition`	HK should be constant
✍	***CHALLENGE #3: Does HK gene look stable? If not, which condition drifts?***		
✍	***Your answer: _____***		
0:25	Print suspect rows	`piv[piv["Sample"] .str.contains(...)]`	Confirm: CT is wrong
Part D: Fix Source File & Re-Visualize (7 min)			
0:27	Open Sample_Data.ods	`# Find Cond 2 Rep 2, # Target Gene row # Replace 35.12 → 22.560 # Save the file`	Fix the spreadsheet!
0:30	Re-run: read + process	`# Re-run cells: # read, fix Gene, avg, # split, merge, group`	Now reads corrected file
0:32	Re-run: scatter	`plt.show()`	Outlier gone!
✍	***CHECKPOINT: Scatter clean? All HK values cluster approx 7–9?***		
Part E: ΔΔCt Calculations (10 min)			
0:34	Step 1: dCT	`piv["dCT"] = ct_target - ct_hk`	Control approx 22.7
0:36	Practice: filter → mean	`cond1_mean = piv[ piv["group"] =="Condition 1"] ["dCT"].mean()`	Same pattern from Part A
0:37	Step 2a: control_mean	`# YOUR CODE HERE` *Hint: piv[piv["group"]=="Control"] ["dCT"].mean()*	Should print 22.749
0:39	Step 2b: ddCT	`piv["ddCT"] = dCT - ctrl_mean`	Control approx 0
0:40	Step 3: 2^(-ddCT)	`# YOUR CODE HERE` *Hint: 2 ** (-piv["ddCT"])*	Control approx 1.0
0:42	View final table	`print(piv[["Sample", "dCT","ddCT", "2^(-ddCT)"]])`	12 rows, match slides
✍	***Q: Why are Control values approx 1.0? What does approx 311 for Cond 2 mean biologically?***		
✍	***Your answer: _____***		
Part F: Visualize Results (8 min)			
0:43	Summarize by group	`summary = piv.groupby(...) .agg(["mean", "std"])`	Cond1 approx 14, Cond2 approx 311
0:45	Bar chart + error bars	`plt.bar(order, means, yerr=stds) plt.show()`	4 bars, Cond 3 tallest
✍	***Q: Which condition has most variability? Why might that be?***		
✍	***Your answer: _____***		
Part G: Statistical Analysis (7 min)			
0:49	Setup: separate groups	`ctrl = piv[..."Control"] ["2^(-ddCT)"]`	Same filter pattern!
0:50	t-test: Cond 1 vs Ctrl	`# YOUR CODE HERE` *Hint: stats.ttest_ind(cond1, ctrl)*	p < 0.05?
0:52	t-test: Cond 2 vs Ctrl	`t, p = stats.ttest_ind( cond2, ctrl)`	p value?
0:54	t-test: Cond 3 vs Ctrl	`# YOUR CODE HERE`	p value?

		Hint: Same pattern as Cond 2	
✍	***Q: Are ALL three conditions statistically significant vs Control?***		
✍	***Your answer: _____***		
✍	***Q: Cond 3 has highest fold change but widest error bars. More significant than Cond 2?***		
✍	***Your answer: _____***		
0:56	Save everything	`plt.savefig("fold.png")` `piv.to_csv("results.csv")`	Files saved!
0:58	Wrap-up	`# Save notebook! # Export as PDF`	Pipeline complete!

Troubleshooting Guide

Stuck? Find your error below:

Problem	Cause	Fix
ModuleNotFoundError: odfpy	odfpy not installed	`!pip install odfpy`
FileNotFoundError	File not uploaded	`# Colab: upload via # folder icon sidebar`
KeyError: column name	Name doesn't match	`print(df.columns.tolist())`
df["Gene"].unique() shows 3 values	Extra space in gene name	`df["Gene"].str.strip() .str.replace(...)`
merge gives 0 rows	Gene typo not fixed before split+merge	`# Fix Gene first, # then re-run`
All dCT identical	Wrong subtraction: HK - Target	`# Must be: # ct_target - ct_hk`
control_mean is NaN	Group name mismatch	`print(piv["group"] .unique())`
2^(-ddCT) approx 1 for Controls	Correct! Controls are the baseline	`# ddCT approx 0 # so 2^0 = 1`
Re-run doesn't show the fix	File not re-uploaded or kernel cached	`# Re-upload .ods # Runtime → Restart # and Run All`
Cond 2 t-test p > 0.05	Bad CT still in data	`# Fix 35.12 → 22.560 # Re-upload + re-run`
plt.show() nothing	Missing magic command	`% matplotlib inline`

ΔΔCt Quick Reference

Step	Formula	Python Code
Step 1	ΔCt = Ct(target) − Ct(HK)	`dCT = ct_target - ct_hk`
Step 2	ΔΔCt = ΔCt − ΔCt(ctrl)	`ddCT = dCT - control_mean`
Step 3	2^(−ΔΔCt)	`2**(-ddCT)`

Expected Values (after fixing all issues)

Group	Avg dCT	Avg ddCT	Avg 2^(-ddCT)
Control	22.75	approx 0	approx 1.0
Condition 1	18.92	−3.83	approx 14
Condition 2	14.48	−8.27	approx 311
Condition 3	12.96	−9.79	approx 933